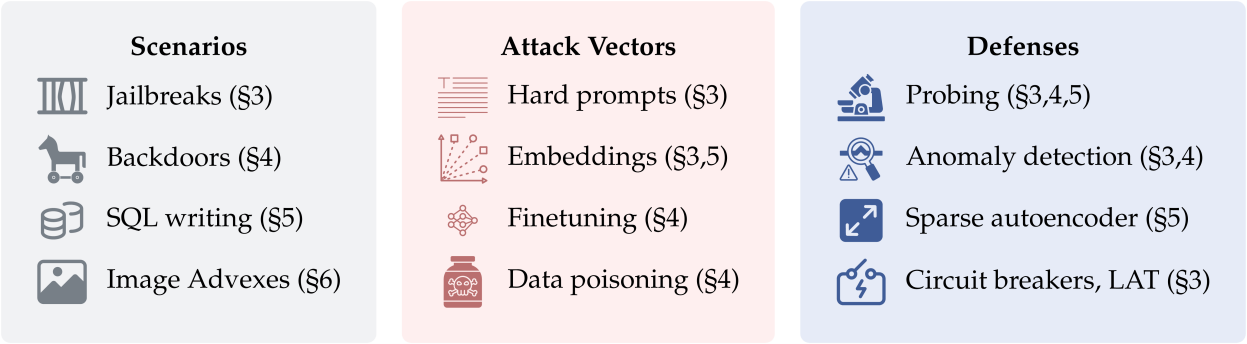

Figure 3: **Overview of the attack and defense settings we study.**

- **Sparse autoencoders (SAEs)** (§5). We find embedding suffixes that let an LLM generate SQL code without triggering monitors meant to detect SQL using SAE latents. However, obfuscation against both SAEs and traditional probes can reduce performance at writing *correct* SQL code. This suggests that a trade-off between an attacker's goals—model behavior and inconspicuous activations—does exist in at least some domains.
- **Exploratory case studies** (§6). We present smaller investigations into additional topics. We show obfuscation attacks against image classifiers (§6.1) and present *Obfuscated Adversarial Training (OAT)*, an approach toward potentially mitigating obfuscated activations by training models to be easier to monitor (§6.2). We also show limits on obfuscation, failing to find activations that are almost orthogonal to natural ones while preserving the original model behavior (§6.3).

Adversarial attacks on latent monitors have been studied before, especially in the image classification domain (see §7). Building on this work, we primarily focus on the generative setting of LLMs with complex model behaviors. Our key contributions are:

1. **Empirical and methodological:** We demonstrate methods to induce obfuscated activations against a wide array of latent monitoring techniques, including sparse autoencoders, OOD detectors, and adversarially trained probes.
2. **Conceptual**: By demonstrating the broad existence of obfuscated activations, we forward the hypothesis that *models can represent concepts and execute behaviors using a broad set of different activations*. That is, neural representations are *highly malleable*: we can reshape activation patterns in a variety of ways, while preserving a network's behavior.

2 Definitions and Background

Obfuscated Activations. When activations induced by an adversarial attack fool a latent space monitor, we call them *obfuscated* against that monitor. More formally, for some adversarial input, x_{adv}, let $\mathcal{R}(x_{\text{adv}})$ represent a model M's latent activations. Then let D refer to some latent-space monitoring process, which outputs real-valued scores $D(\mathcal{R}(x))$ for how strongly a certain concept, such as harmfulness, is present in the input x or the generation. Then $\mathcal{R}(x_{\text{adv}})$ is an obfuscated activation if $D(\mathcal{R}(x_{\text{adv}}))$ is low even though the concept D is meant to detect is clearly present. We study both how often obfuscated activations appear as a side effect of traditional attacks and how easily they can be induced deliberately through obfuscation attacks.

Obfuscation Attacks. An obfuscation attack is an attack that aims to induce obfuscated activations by optimizing a joint objective consisting of a behavioral loss term and an obfuscation loss:

$$\min \quad \mathcal{L}_{\text{beh}}(M(x)) + \lambda_{\text{obf}}\mathcal{L}_{\text{obf}}(\mathcal{R}(x)) \tag{1}$$

For example, $\mathcal{L}_{\text{beh}}$ can be a cross-entropy loss designed to make the model output a harmful string. For the obfuscation loss, we often use $\mathcal{L}_{\text{obf}} = D$ on inputs with the concept, i.e., we directly attempt to fool D. We also explore transfer, i.e., when does obfuscating against a monitor D' fool D. Finally, λ_{obf} is some real non-negative value used for balancing the two terms.

2.1 Monitoring methods

We consider latent space monitors D that take in residual stream activations of the model M. Given activations $\mathcal{R}(x)$, the monitor outputs a real-valued score $D(\mathcal{R}(x))$. We consider two approaches, *input-based* and *generation-based* monitors. Input-based monitors use only the activations on the final prompt token, $\mathcal{R}(x)[\texttt{len}(x) - 1]$, as is common in latent probing methods [Burns et al., 2022, Marks and Tegmark, 2024, Kirch et al., 2024]. Generation-based monitors use all tokens of the generation, $\mathcal{R}(x)[\texttt{len}(x) :]$. Most of our monitors separately consider activations at multiple layers of the model and aggregate scores over layers and sequence dimensions by taking a mean.

Supervised probes. We test different supervised binary probes trained to detect a certain concept, such as harmfulness or SQL code. The simplest ones are mean difference probes, which use the difference vector between the mean of positive training samples and the mean of negative training samples as a linear classifier. We also test linear classifiers trained using logistic regression, as well as MLP classifiers.

OOD detection. A complementary approach to probing is latent-space anomaly or out-of-distribution (OOD) detection. We train an OOD detector on examples of "typical" usage, and use its OOD score to classify new samples. We test a Mahanalobis distance-based detector [Mahalanobis, 1936, Podolskiy et al., 2021, Chen et al., 2022], which fits a multivariate Gaussian to activations of negative samples and uses the log-likelihood as an OOD score. We also test Variational Autoencoders (VAEs) [Kingma and Welling, 2013] with the ELBO as the OOD score. See Appendix E for details on these methods. In the backdoor case study (§4), we also test state-of-the-art OOD detectors specifically developed for backdoor detection [Ma et al., 2023, Mo et al., 2024].

Sparse autoencoders. Sparse autoencoders (SAEs) have become a popular tool in mechanistic interpretability [Cunningham et al., 2023, Templeton et al., 2024, Gao et al., 2024, Lieberum et al., 2024]. They learn features in an unsupervised way. Each feature is represented by a linear map followed by a non-linearity. This then allows for a human, or some other process, to flag risky examples based on what their SAE latents suggest. In §5, we train linear probes on SAE latents and also test the latents themselves as classifiers.

We also study Circuit Breakers [Zou et al., 2024] and Latent Adversarial Training [Casper et al., 2024, Sheshadri et al., 2024] in §3.4, which are not themselves monitoring methods but share certain similarities, as we discuss in the relevant section.

2.2 Evaluation

In this paper, the attacker has a dual goal: to induce some target behavior in the model *while* inducing obfuscated activations. This raises a possible tradeoff. For example, jailbreaking a model more reliably leads to more consistently harmful outputs, which might make it more difficult to fool a harmfulness classifier. We thus always evaluate both whether the intended behavior was achieved and whether activations are obfuscated.

Evaluating behavior. We use StrongREJECT [Souly et al., 2024] to evaluate whether a model output is harmful. In §5, the intended behavior is instead to generate correct SQL code, and we evaluate this by running the model generation against an SQL database and checking that the entries returned are correct.

Evaluating obfuscation. To quantify whether a set of activations fool a monitor, we use recall at a 1% false positive rate (FPR). In other words, we pick a score threshold so that the monitor classifies at most 1% of negative (e.g., benign) samples as positive (e.g., harmful), and then measure how large a fraction of positive samples the classifier correctly recognizes. Unlike AUROC, this metric focuses on the low FPR regime, which is the most relevant one for many applications (e.g., a chatbot should not refuse benign requests too often). See Appendix F for more detailed discussion.

We compute confidence intervals by bootstrapping; i.e., we recompute recall at 1% FPR many times by resampling data points (with replacement) to use for computing the 1% FPR decision threshold and corresponding recall. In a few cases, recall is highly sensitive to the specific FPR we pick (e.g., the recall at 2% FPR might be a lot higher than the 1% one we report). This usually results in large error bars, but we additionally mention such cases in the text.

3 Case Study: Detecting Harmfulness

Robustly preventing jailbreaking attacks has proven to be a challenging problem—most defenses can quickly be broken using new attacks [Shayegani et al., 2023, Jin et al., 2024, Yi et al., 2024, Li et al., 2024]. Latent-space defenses aim to tackle the problem at its root: preventing or recognizing representations necessary for harmful outputs might defend against any potential attack [Zou et al., 2024]. However, this hope assumes that there is a compactly describable set of such harmful representations that can be reliably recognized. In this section, we scrutinize this key assumption. We address the following questions:

(1) Can existing latent-space monitors reliably detect harmful inputs and outputs against standard jailbreaking methods? (§3.2)

(2) Can obfuscation attacks induce harmful outputs via inconspicuous activations and thus bypass latent-space harmfulness monitors? (§3.3, §3.4)

(3) How many different ways can a model represent harmfulness and what does this mean for adversarial training of probes? (§3.5)

3.1 Setup

We train probes to distinguish between benign and harmful text. Our dataset consists of benign inputs with benign generations and harmful inputs with teacher-forced harmful generations. We